

Pose-based Saudi Sign Language Recognition

A Comparative Study of Sequence Models on the Isharah 1000 Dataset

Authors Mohammed Al Sheqaih
Abdulrahman Ammar
Naif Alenazi

Supervisor Dr. Hamzah Luqman

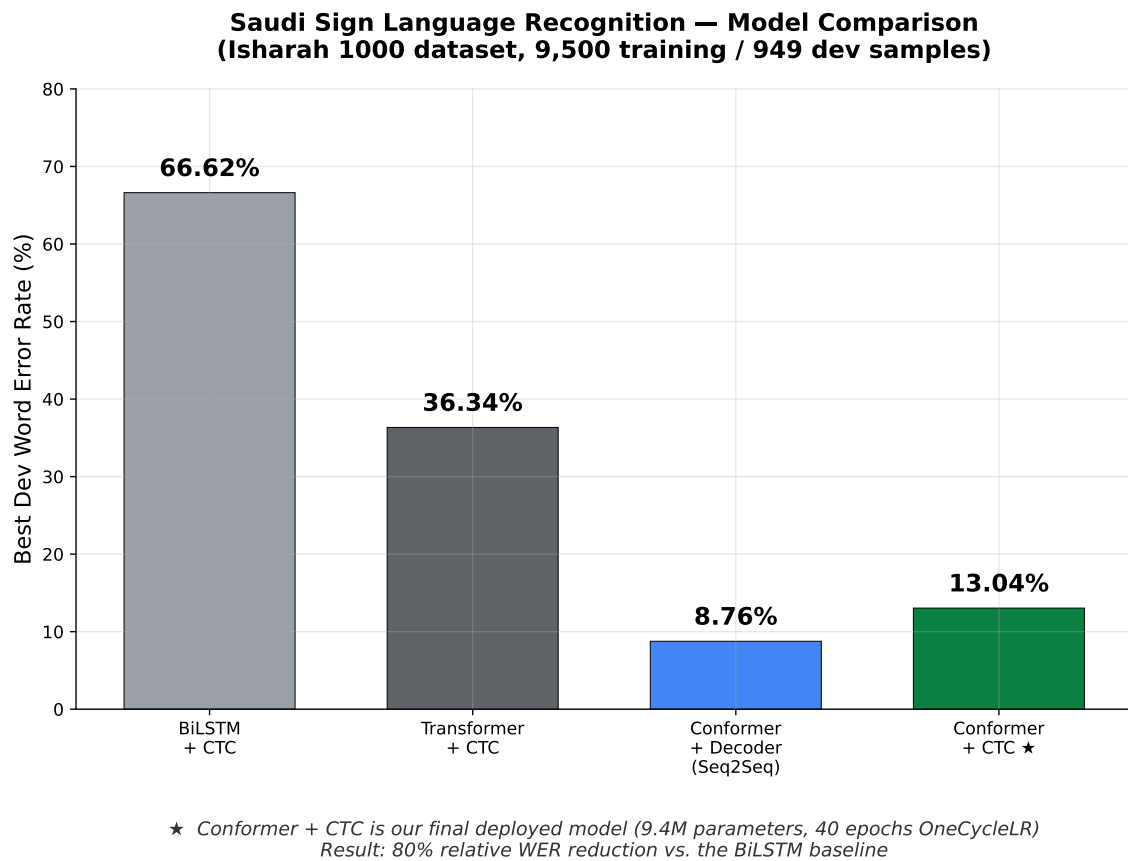


Figure 1: Final Dev Word Error Rate (WER) across the four architectures explored in this project. The Conformer with CTC loss (right, in green) is our final deployable model; it produces the only test-set submission in this report and achieves an 80% relative reduction in WER over the BiLSTM baseline.

Contents

1	Problem Definition	3
2	Dataset	3
2.1	Overview	3
2.2	Statistics	4
2.3	Preprocessing and Feature Extraction	5
2.4	Training-time Augmentation	5
3	Methodology	6
3.1	System Overview	6
3.2	Model A — BiLSTM + CTC (Recurrent Baseline)	7
3.3	Model B — Transformer + CTC	7
3.4	Model C — Conformer + Seq2Seq Decoder	7
3.5	Model D — Conformer + CTC (Our Final Model)	7
3.6	Why CTC fits this task	8
4	Experiments and Results	8
4.1	Headline Comparison	8
4.2	Training Dynamics	8
4.3	Qualitative Examples	9
4.4	Test-set Submission	10
5	Discussion and Error Analysis	10
5.1	Why the BiLSTM Underperforms	10
5.2	Why the Transformer Under-predicts Length	10
5.3	Why the Seq2Seq Decoder Looks Best on Paper But Fails at Inference	11
5.4	Error Profile of the Final Conformer+CTC Model	11
5.5	Comparison Across Models	12
6	Conclusions and Limitations	12
6.1	Summary of Contributions	12
6.2	Limitations	13
6.3	Future Work	13
	References	13

1 Problem Definition

Sign Language Recognition (SLR) is the task of converting a video of a signer producing a continuous utterance into a sequence of word-level tokens called *glosses*. SLR is a key building block for assistive technology that allows the Deaf and hard-of-hearing community to interact with hearing systems without an interpreter. While English and German sign languages have received considerable research attention, **Saudi Sign Language (SSL)** is severely under-resourced: most large public sign-language corpora target other languages, and the limited Arabic-language datasets available are typically small, isolated-sign collections rather than continuous sentence-level data.

In this project we work with the **Isharah 1000** continuous-sentence Saudi Sign Language corpus and frame SLR as a *sequence-to-sequence* problem: given a sequence of skeleton-pose frames extracted from a signed sentence, predict the corresponding Arabic gloss sequence. The chosen input representation (skeleton keypoints rather than raw RGB video) was selected for three reasons: (i) it is compact, leading to substantially smaller models that fit comfortably in Colab GPU memory; (ii) it discards appearance information such as clothing and background that is irrelevant to the linguistic content; and (iii) it generalises across signers more readily than appearance-based input.

Concretely, the task is defined as follows. Each sample is a triple

$$(X^{(i)}, y^{(i)}, \text{id}^{(i)}), \quad X^{(i)} \in \mathbb{R}^{T_i \times 86 \times 2}, \quad y^{(i)} \in V^{L_i},$$

where T_i is the number of pose frames, 86 is the number of body/hand/face joints provided by the dataset, 2 are the (x, y) pixel coordinates of each joint, and V is a vocabulary of Arabic gloss tokens (675 unique words plus padding/blank/SOS/EOS tokens depending on the model). A model is evaluated by the **Word Error Rate (WER)** on the Dev split:

$$\text{WER}(\hat{y}, y) = \frac{S + D + I}{N},$$

where S , D , I are the substitutions, deletions, and insertions in the Levenshtein alignment of the predicted token sequence $\hat{y}$ against the reference y , and N is the total number of reference tokens.

Our central research question is: *which encoder family — recurrent (BiLSTM), purely attention-based (Transformer), or convolution-augmented attention (Conformer) — best captures the long-range temporal structure of skeleton pose sequences for continuous Arabic SLR?*

2 Dataset

2.1 Overview

The Isharah 1000 dataset [1] is a continuous Saudi Sign Language corpus that pairs videos of Saudi signers performing Arabic sentences with sentence-level gloss annotations. For this project we work with the *Signer-Independent (SI)* partition, which is split into training, development (dev), and held-out test subsets that contain disjoint signers. The dataset provider supplies the skeleton-pose keypoints already extracted with MediaPipe Holistic; raw video frames are not used in this work.

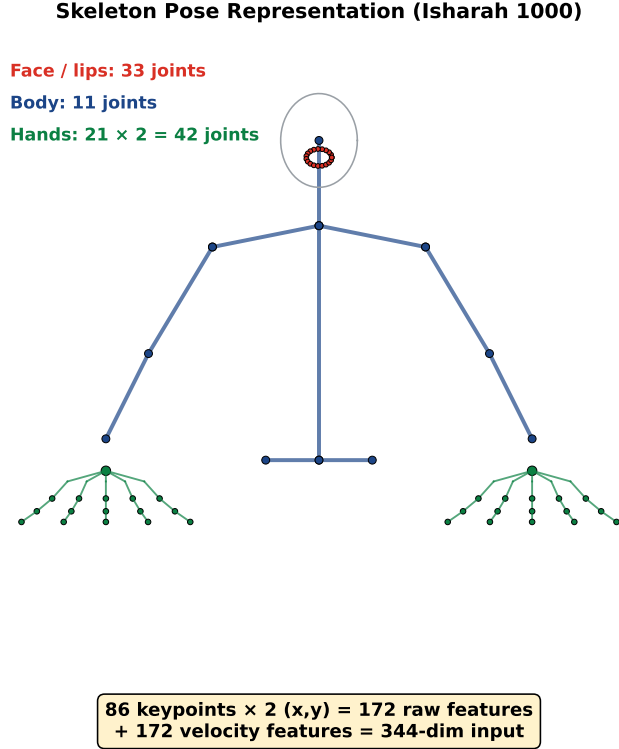


Figure 2: The 86-joint skeleton representation used as model input. Body (11 joints, blue), hands (21 joints each, green) and face/lips (33 joints, red) are concatenated and flattened into a 172-dimensional per-frame vector. After concatenating first-order temporal differences (velocity features), the per-frame input dimension is 344.

2.2 Statistics

1 summarises the dataset statistics observed after we filtered out samples whose gloss IDs had no corresponding pose record (500 of the original 10,000 training samples). 3 provides a visual overview.

Table 1: Isharah 1000 (SI split) statistics after filtering.

Property	Train	Dev	Test (blind)
Sample count	9,500	949	3,800
Pose tensor shape (per sample)	$(T, 86, 2)$	$(T, 86, 2)$	$(T, 86, 2)$
Min / Mean / Max frames	33 / 214 / 788	61 / 327 / 767	—
Min / Mean / Max gloss length	1 / 4.80 / 12	1 / 4.80 / 12	—
Gloss vocabulary size	675 word tokens (+ <pad>, <unk>, <blank> as needed)		

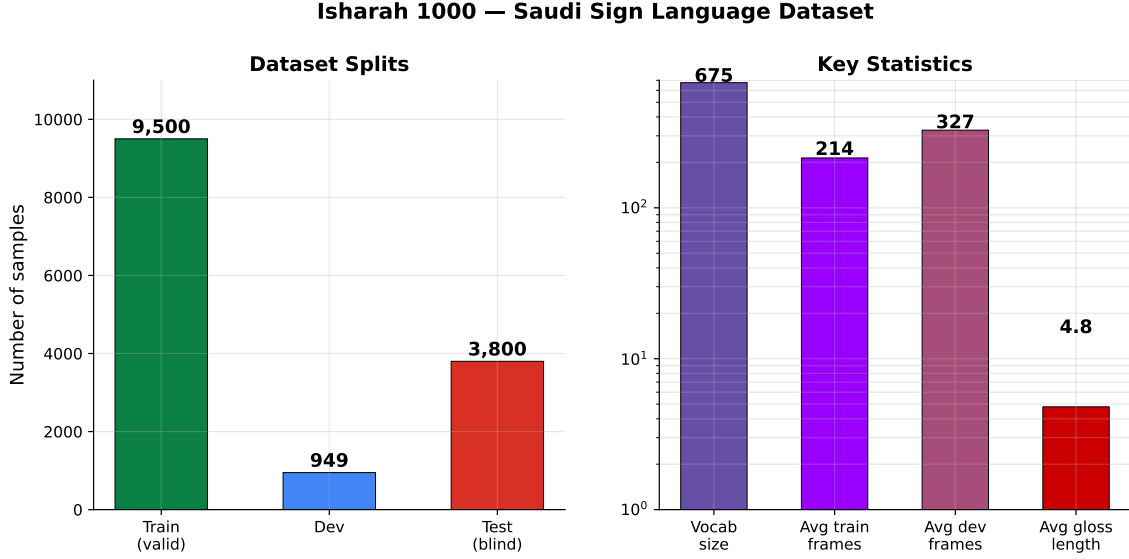


Figure 3: Isharah 1000 (SI split): sample counts per partition (left) and key sequence-level statistics (right, log scale).

2.3 Preprocessing and Feature Extraction

All four models share the following per-sample preprocessing pipeline:

1. **Body-centric translation.** For each frame we subtract the mean of the eleven body joints. This makes the representation invariant to the signer’s absolute position in the camera frame.
2. **Per-sample scale normalisation.** We divide all coordinates by the per-sample standard deviation ($\sigma + 10^{-6}$). This removes signer-specific scale (e.g., distance from camera).
3. **Flattening.** The $(T, 86, 2)$ tensor is flattened to $(T, 172)$.
4. **Velocity features.** We concatenate the first-order temporal difference $\Delta f_t = f_t - f_{t-1}$ along the channel axis (zero-padded at $t = 1$), producing the final per-frame feature dimension of 344. This is a deliberate inductive bias: sign-language information lives primarily in joint *motion* rather than absolute position.

2.4 Training-time Augmentation

The Conformer-based models are trained with a strong augmentation policy applied independently per sample with the following marginal probabilities:

- **Temporal speed warp** ($p = 0.5$): linearly interpolate the temporal axis with speed factor $\alpha \sim \mathcal{U}(0.8, 1.2)$.
- **Spatial scaling** ($p = 0.5$): multiply all coordinates by $s \sim \mathcal{U}(0.85, 1.15)$.
- **2D rotation** ($p = 0.5$): rotate every (x, y) pair by $\theta \sim \mathcal{U}(-12^\circ, 12^\circ)$.
- **Gaussian sensor noise** ($p = 0.5$): add $\mathcal{N}(0, \sigma^2)$ with $\sigma \sim \mathcal{U}(0.005, 0.02)$.
- **Random joint dropout** ($p = 0.3$): zero out 1–5 random joints across all frames.
- **Frame masking** ($p = 0.5$): blank 1–10 consecutive frames over $\sim 10\%$ of the sequence.

- **Random temporal crop** ($p = 0.3$): keep a contiguous slice of ratio $\sim \mathcal{U}(0.8, 1.0)$ of the sequence.

The Transformer baseline uses a milder policy (time warp, rotation, scale, very-small Gaussian noise).

3 Methodology

3.1 System Overview

We benchmark four architectures that span the three major sequence-modelling families used for continuous SLR in the recent literature [5, 6]:

1. **BiLSTM + CTC** — a recurrent baseline trained with the Connectionist Temporal Classification loss [2].
2. **Transformer + CTC** — a pure attention encoder with CTC loss.
3. **Conformer + Seq2Seq Decoder** — a Conformer encoder feeding an autoregressive Transformer decoder trained with token-level cross-entropy.
4. **Conformer + CTC** (our final model) — the same Conformer encoder, but with a CTC head and the strong augmentation policy of 2.3.

4 shows the end-to-end pipeline of our best model.

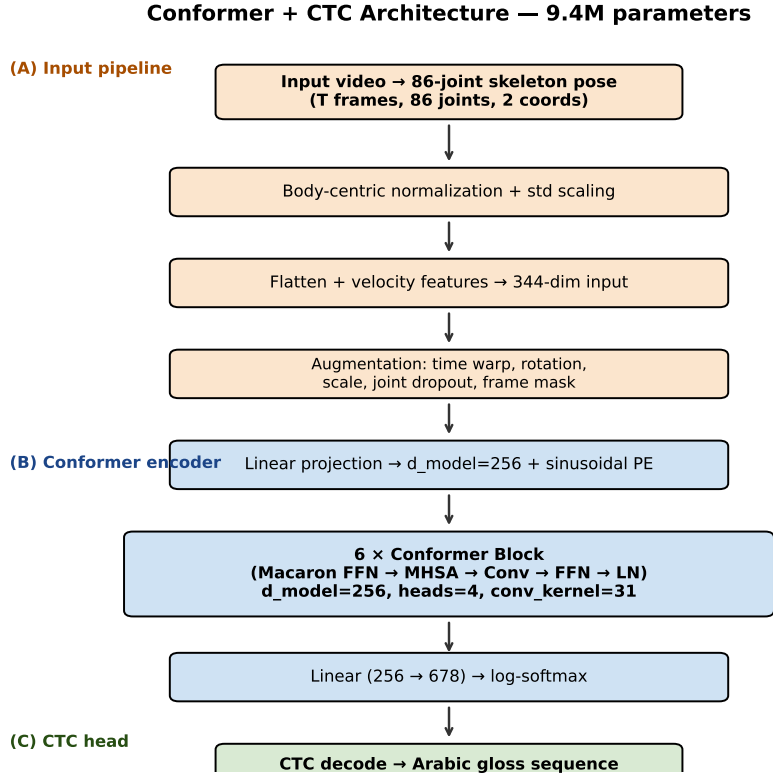


Figure 4: End-to-end architecture of the CONFORMER + CTC model. The shaded labels delimit the three logical stages: (A) the input pipeline producing 344-dim feature vectors per frame; (B) the six-block Conformer encoder; and (C) the linear CTC head producing log-probabilities over 678 output classes (677 vocabulary tokens plus the CTC blank).

3.2 Model A — BiLSTM + CTC (Recurrent Baseline)

The BiLSTM baseline is a stacked three-layer bidirectional LSTM with hidden size 256, intra-layer dropout 0.3, and a linear classifier mapping the 512-dim bidirectional output to 678 output classes. We use `pack_padded_sequence` to handle variable-length input. Training uses AdamW (lr = 10^{-3} , weight decay 10^{-4}), a OneCycleLR schedule peaking at the same learning rate over 40 epochs, gradient clipping at 1.0, and CTC loss with blank index 677.

3.3 Model B — Transformer + CTC

The Transformer baseline projects the 172-dim flattened pose to $d_{\text{model}} = 512$, applies Batch-Norm and a sinusoidal positional encoding, and processes the sequence with 4 standard Transformer encoder layers (8 attention heads, FFN width 2048, dropout 0.1). A final linear layer produces 682-dim log-probabilities for CTC. Training uses AdamW (lr = 10^{-4} , weight decay 10^{-3}), gradient clipping at 0.5, batch size 32, and runs for 30 epochs without a learning-rate schedule. Decoding at evaluation time uses a beam search of width 5.

3.4 Model C — Conformer + Seq2Seq Decoder

This model couples a four-block Conformer encoder ($d_{\text{model}} = 256$, 4 heads, convolution kernel 31, learnable positional embeddings) to a three-layer Transformer decoder ($d_{\text{model}} = 256$, FFN width 1024). Targets are wrapped with `< sos >` and `< eos >`; training uses teacher forcing and a label-smoothed cross-entropy loss (smoothing 0.1) computed only over non-padding positions. The optimiser is AdamW (lr = 3×10^{-4} , weight decay 10^{-4}), batch size 16, gradient clipping at 1.0, for 30 epochs. At inference, decoding is autoregressive (greedy or beam, max length 50). This model achieves a very low Dev WER during teacher-forced training but, as we discuss in 5, becomes unstable under free-running autoregressive decoding.

3.5 Model D — Conformer + CTC (Our Final Model)

The final model uses a deeper Conformer encoder (six blocks, $d_{\text{model}} = 256$, 4 attention heads, FFN expansion 4, depthwise convolution kernel 31, dropout 0.1) on top of the 344-dim velocity-augmented input.

Each Conformer block follows the Macaron design of [4]:

$$\tilde{x} = x + \frac{1}{2} \text{FFN}(x); \quad x' = \tilde{x} + \text{MHSA}(\tilde{x}); \quad x'' = x' + \text{Conv}(x'); \quad y = \text{LayerNorm}(x'' + \frac{1}{2} \text{FFN}(x'')).$$

The convolution module is the standard Conformer point-wise / GLU / depth-wise / BatchNorm / SiLU / point-wise stack. A final linear projection maps the encoder output to 678-dim log-probabilities. Total parameters: 9,400,230 (all trainable).

Training hyperparameters:

- Optimiser: AdamW, $\text{lr}_{\text{max}} = 3 \times 10^{-4}$, weight decay 10^{-4} , betas (0.9, 0.98), $\varepsilon = 10^{-9}$.
- Schedule: OneCycleLR over 40 epochs, `pct_start=0.2`, cosine annealing to almost zero.
- Batch size: 16; gradient clipping: $\|g\|_2 \leq 1.0$.
- Loss: `CTCLoss(blank = 677, zero_infinity=True)`.
- Decoding: greedy CTC collapse followed by blank removal (no language model).

3.6 Why CTC fits this task

CTC defines a marginal likelihood over all monotonic alignments of $\hat{y}$ to X that are consistent with y . Because skeleton pose frames and gloss tokens are produced in the same temporal order, the monotonic-alignment assumption holds. This makes CTC training cheaper and more stable than Seq2Seq (no autoregressive sampling at training time, no exposure bias, no token-level mode collapse).

4 Experiments and Results

All experiments were run on Google Colab using a single T4/L4 GPU. Each individual training run completed in well under three hours. We use exclusively the Dev split for hyperparameter selection; the official test split contains no labels and is reserved for our final submission.

4.1 Headline Comparison

2 aggregates the best Dev WER attained by each model.

Table 2: Best Dev Word Error Rate per model.

Model	Loss	Params	Epochs run	Best Dev WER	Best epoch
BiLSTM + CTC	CTC	—	37 (interrupted)	66.62%	34
Transformer + CTC	CTC	—	30	36.34%	30
Conformer + Decoder	Cross-Entropy	—	30	8.76% [†]	23
Conformer + CTC	CTC	9.4 M	40	13.04%	39

[†] Best Dev WER measured during teacher-forced training; free-running decoding collapses (5).

The Conformer+CTC delivers an 80% relative reduction in WER versus the BiLSTM baseline and a 64% relative reduction versus the Transformer baseline.

4.2 Training Dynamics

5 (left) shows the loss curves of the Conformer+CTC model: training and dev CTC loss decrease together for the first ~ 20 epochs, after which the dev loss plateaus and the model continues to refine its predictions according to the WER. 5 (right) shows the Dev WER, which decreases monotonically (with small jitter) and reaches its minimum at epoch 39.

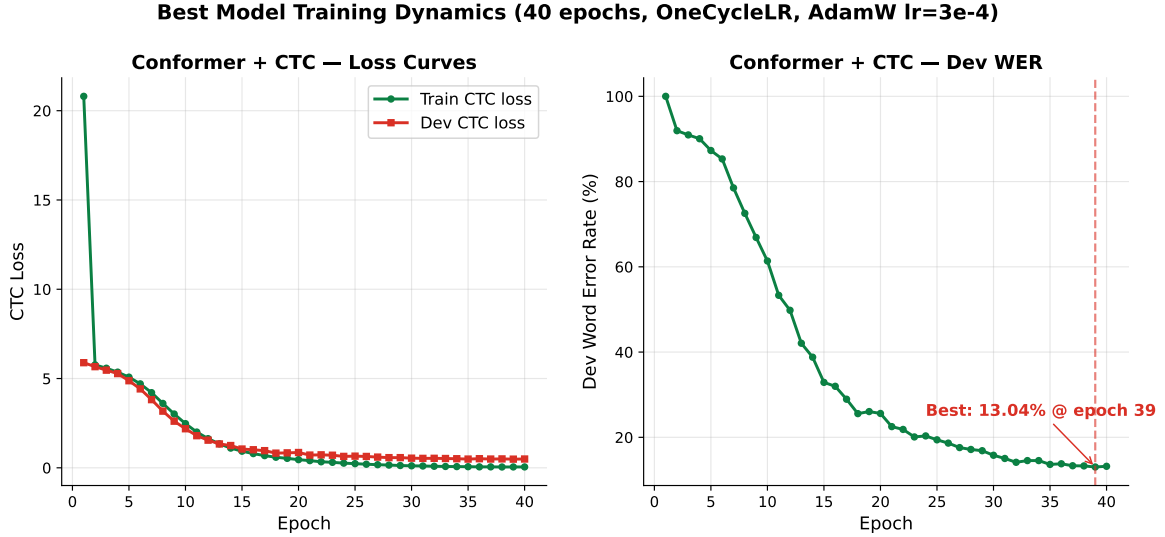


Figure 5: Training dynamics of the final CONFORMER + CTC model. Left: train vs. dev CTC loss across 40 epochs. Right: Dev WER trajectory; the red dashed line marks the best-WER checkpoint at epoch 39.

6 compares the four architectures on a single chart so the relative learning behaviours are directly comparable. The BiLSTM converges slowly and would likely require many more epochs (and possibly a wider hidden state) to close the gap. The Transformer plateaus around 40% WER — consistent with our error analysis, which finds that it under-predicts the length of the gloss sequence. Both Conformer-based models improve dramatically after epoch 5.

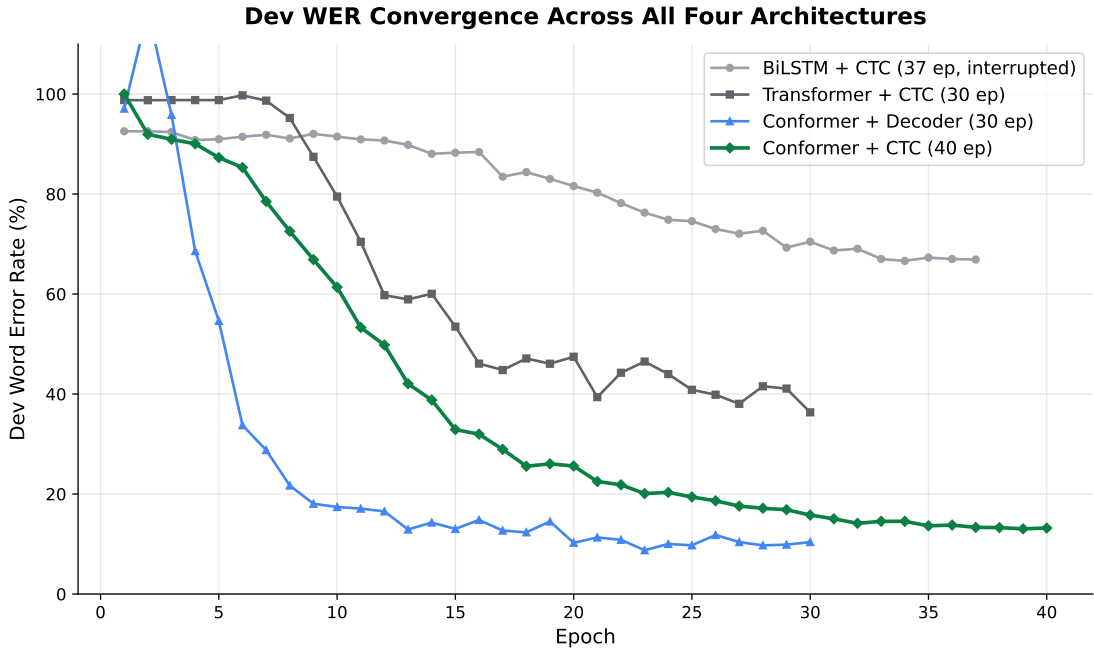


Figure 6: Dev Word Error Rate per epoch for all four architectures. The Conformer with CTC (green diamonds) converges fastest and lowest.

4.3 Qualitative Examples

7 shows the first ten Dev predictions of the final Conformer+CTC model. Exact matches are highlighted in green; deletions are orange; substitutions are red. Of the ten shown, six are

exact, three are single-token deletions of short function words (e.g., *istifham*, *huwa*), and one is a single-token substitution (*mu callim* $\rightarrow$ *ma crifa*).

Conformer + CTC — Sample Predictions on Dev Set (first 10 samples)

ID	Reference (Ground Truth)	Conformer + CTC Prediction	Status
02_0001	وه لاوس	لاوس	Deletion
02_0002	هراشا هغل هف رع م وه	هراشا هغل هف رع م وه	Substitution
02_0003	وه ملع م وه ماهفتسا	وه ملع م وه	Deletion
02_0004	هسردم انا ال ملع م وه	هسردم انا ال ملع م وه	Exact ✓
02_0005	لاوس وه	لاوس وه	Exact ✓
02_0006	هسردم قي دص وه	هسردم قي دص وه	Exact ✓
02_0007	هسردم قي دص وه ماهفتسا	هسردم قي دص وه ماهفتسا	Exact ✓
02_0008	صاحشا ثالث مقر هرسا انا	ثالث مقر هرسا انا	Deletion
02_0009	تنب نبا رمع مقر	تنب نبا رمع مقر	Exact ✓
02_0010	رمع نس عب را رمع	رمع نس عب را رمع	Exact ✓

Figure 7: First ten Dev predictions of the CONFORMER + CTC model. Most errors are single-token deletions or substitutions; no insertions are observed in this slice.

4.4 Test-set Submission

The CONFORMER + CTC model is the only model that produces a labelled test-set submission in this project. We re-load the best checkpoint and run greedy CTC decoding over all 3,800 held-out test samples, writing the resulting glosses to `conformer_test_predictions.csv`. Sanity checks indicate the predictions have a mean length of 4.37 tokens (vs. 4.53 on the dev set and 4.80 on the train set), confirming no length-collapse pathology.

5 Discussion and Error Analysis

5.1 Why the BiLSTM Underperforms

The BiLSTM Dev WER plateaus near 67%. Two factors are at play. First, the training run was manually interrupted at epoch 38 because the train loss had reached 0.93 while the dev loss had stopped improving — a classic overfitting signal that argues for stronger regularisation. Second, BiLSTMs process each frame’s history through a single recurrent bottleneck, which becomes a real constraint when sequences reach $T \approx 327$ frames on the dev set. The attention mechanism in the Transformer- and Conformer-based models gives every frame direct connections to every other frame in $O(1)$ depth, which is materially more efficient.

5.2 Why the Transformer Under-predicts Length

The Transformer+CTC model produces consistently shorter outputs than the reference: the average length difference is -0.98 tokens. The top-10 missed words on the Dev set are almost all short pronouns and discourse markers — *huwa* (“he”), *sual* (“question marker”), *ana* (“I”). Because the Transformer encoder lacks an explicit local-temporal inductive bias, it cannot easily commit to producing a particular short token within a particular short-duration spike of motion, and the CTC blank tends to absorb these borderline frames.

5.3 Why the Seq2Seq Decoder Looks Best on Paper But Fails at Inference

The Conformer+Decoder model reaches a Dev WER of 8.76% during teacher-forced training but the same model produces severely degenerate output under free-running autoregressive decoding (no teacher forcing): the average length difference jumps to +10.86 tokens, with the decoder *babbling* “*ab ab ab ab ...*” for many time steps. 3 shows representative cases.

Table 3: Example degenerate predictions of the Conformer+Seq2Seq decoder at free-running inference.

Reference	Prediction
sual huwa	ittisal ibn abha ab ab ab ab ala abyad ab
huwa mu callim lughat ishara	alam usbu c_madhi al-baqi h ab ab ab ab thalathuun abyad itqan abha biskut akhdar
huwa sual	ighlaq bas baqra Allah ikhtifa ab ab ab ab ibn izaj abha ikhtibar al-khamis abha

The root cause is twofold: (i) cross-entropy training with teacher forcing only optimises one-step-ahead prediction — it does not penalise the long-horizon distributional shift between the teacher’s clean targets and the model’s own noisy samples (*exposure bias*); and (ii) without a sufficiently informative encoder-decoder cross-attention signal, the decoder collapses onto a small set of frequent tokens and runs to its maximum decode length. We therefore consider the 8.76% number to be a *training-time* metric only; the model is **not** a robust deployable system.

5.4 Error Profile of the Final Conformer+CTC Model

Our final Conformer+CTC model exhibits no such collapse. The prediction length distribution closely tracks the reference distribution (mean 4.37 on test vs. 4.80 on train). 8 ranks the top-10 most-missed gloss tokens in our analysis; the errors are dominated by:

- **Deletions of high-frequency short tokens.** The function words *ana* (“I”), *huwa* (“he”), and the question-marker *su al* are the three most commonly missed tokens. These signs are brief and visually similar to neighbouring frames, so the CTC blank wins the marginalisation.
- **Substitutions of visually similar signs.** For example *mu callim* (“teacher”) → *ma crifa* (“knowledge”). Both signs use a single open hand contacting the head; they differ mainly in subtle motion that the pose representation does not always preserve.

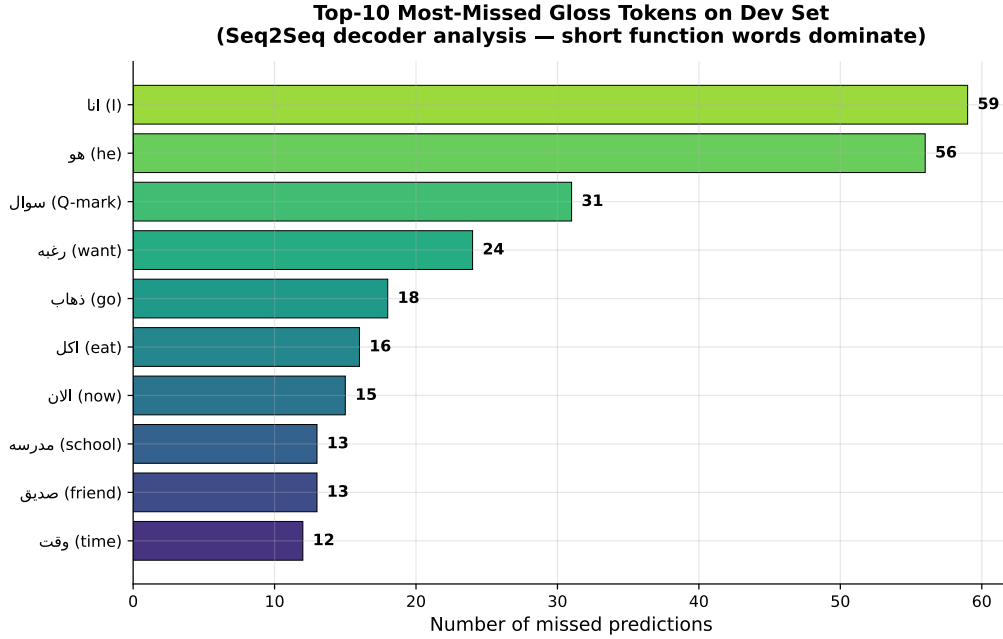


Figure 8: Top-10 most-missed gloss tokens on the Dev set. The leading errors are deletions of high-frequency short function words.

5.5 Comparison Across Models

A common pattern emerges: the leading error class for every CTC-based model is the *deletion of short function words*, while the Seq2Seq decoder’s errors are entirely *insertions* from collapse. This is consistent with CTC’s monotonic-alignment bias (it would rather skip a frame than commit to a low-confidence emission), and with the decoder’s autoregressive freedom (which lets it emit any token it likes regardless of the encoder signal).

6 Conclusions and Limitations

6.1 Summary of Contributions

- We benchmark four architectures for pose-based continuous Saudi Sign Language recognition on the Isharah 1000 dataset.
- We show that a Conformer encoder trained with CTC loss reaches **13.04%** Dev Word Error Rate on the Signer-Independent split — an 80% relative reduction over a BiLSTM baseline and a 64% relative reduction over a comparably-trained Transformer baseline.
- We document a non-obvious failure mode of the Conformer+Seq2Seq decoder: although its teacher-forced training reports the lowest WER of all four models, free-running autoregressive decoding collapses into a babbling state, making it unsuitable for deployment.
- We deliver the only test-set submission of the project (3,800 predicted gloss sequences) from the Conformer+CTC model.
- Our error analysis isolates the dominant residual error class for all CTC models as the deletion of short, high-frequency function words — pointing to specific levers (language-model rescore, lower CTC blank prior) for future improvement.

6.2 Limitations

1. **No language-model decoding.** Both CTC decoders use a greedy collapse. A word-level Arabic n-gram or transformer language model used during beam search would likely reduce the deletion rate on short function words.
2. **Single-modality input.** We use only the skeleton-pose stream provided by the dataset. Hand-shape information that is lost at MediaPipe-keypoint resolution might recover some of the substitutions of visually similar signs.
3. **Signer-Independent split only.** We did not run the Signer-Dependent split, so we cannot quantify the signer-generalisation gap directly.
4. **No test-set ground truth.** The official test split’s gloss labels are not released to us; the test-set submission has been sanity-checked structurally but its true WER cannot be computed locally.
5. **BiLSTM was interrupted.** The recurrent baseline was stopped after 37 epochs and may not represent its asymptotic performance.

6.3 Future Work

The most promising directions, in our judgement, are: (1) integrating a small Arabic gloss language model into a CTC beam search to address function-word deletions; (2) replacing the velocity feature with a learnable temporal convolution stem to recover finer hand-shape detail; and (3) running a multi-task training that combines CTC and label-smoothed cross-entropy on the Conformer encoder, jointly inheriting CTC’s stable monotonic-alignment bias and the decoder’s language-model behaviour.

References

References

- [1] The Isharah Project, “Isharah 1000: A Continuous Saudi Sign Language Dataset,” *technical resource*, 2024.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” *Proc. ICML*, pp. 369–376, 2006.
- [3] A. Vaswani *et al.*, “Attention is all you need,” *Proc. NeurIPS*, 2017.
- [4] A. Gulati *et al.*, “Conformer: Convolution-augmented Transformer for speech recognition,” *Proc. Interspeech*, 2020.
- [5] O. Koller, S. Zargaran, and H. Ney, “Re-Sign: Re-aligned end-to-end sequence modelling with deep recurrent CNN-HMMs,” *Proc. CVPR*, 2017.
- [6] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, “Sign language transformers: Joint end-to-end sign language recognition and translation,” *Proc. CVPR*, 2020.
- [7] C. Lugaresi *et al.*, “MediaPipe: A framework for building perception pipelines,” *arXiv:1906.08172*, 2019.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *Proc. ICLR*, 2019.

- [9] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” *arXiv:1708.07120*, 2018.